

Rapport de Travaux Pratiques

Déploiement et Manipulation de Données sur Hadoop HDFS

Module : Big Data & Systèmes Distribués

Étudiant : Hashimi

Identifiant : c-21454

Année universitaire 2025–2026

Juin 2026

1. Introduction

Le présent rapport documente la réalisation d'un travail pratique portant sur l'architecture Hadoop et le système de fichiers distribué HDFS (Hadoop Distributed File System). L'objectif pédagogique consiste à déployer un mini-cluster Hadoop containerisé, à ingérer un jeu de données volumineux, puis à analyser les mécanismes fondamentaux du Big Data : découpage en blocs, réplication, tolérance aux pannes et administration du cycle de vie du cluster.

Le corpus expérimental repose sur le dataset NYC Taxi Trip (fichier CSV d'environ 562 Mo), déployé au sein d'une infrastructure Docker Compose composée d'un NameNode et de deux DataNodes. L'ensemble des manipulations a été automatisé par des scripts Bash et PowerShell fournis dans le dépôt `hadoop-tp-cluster`.

1.1 Objectifs du travail pratique

- Analyser et lancer un cluster Hadoop via Docker Compose.
- Ingérer et consulter des données massives dans HDFS.
- Étudier le découpage en blocs et la réplication des données.
- Manipuler les flux de données et les opérations avancées (fusion, permissions, append).
- Maîtriser l'arrêt, la persistance et la maintenance sécurisée du cluster.

1.2 Environnement expérimental

L'environnement repose sur Docker Compose (version 3.8), les images `bde2020/hadoop-namenode` et `bde2020/hadoop-datanode` (Hadoop 3.2.1, Java 8), ainsi que les fichiers `hadoop.env` et `tp_config.env`. Le cluster est nommé `hashimi-c21454` et utilise l'espace HDFS `/user/hashimi/c21454/`. L'interface Web du NameNode est accessible sur `http://localhost:9870`.

2. Exercice 1 — Analyse et lancement du cluster

2.1 Analyse du fichier docker-compose.yml

Le fichier `docker-compose.yml` décrit une architecture HDFS minimale mais représentative d'un déploiement distribué : un service namenode (nœud maître) et deux services datanode1 et datanode2 (nœuds esclaves). Chaque service charge les variables d'environnement depuis `hadoop.env` et monte le répertoire local `data` vers `/tp/data`.

Concernant l'exposition des ports, le NameNode publie les ports 9870 (interface Web) et 9000 (protocole HDFS). Les DataNodes exposent respectivement 9864:9864 pour `datanode1` et 9865:9864 pour `datanode2`. Ce second mapping est une pratique standard lorsque plusieurs conteneurs utilisent le même port interne (9864/tcp) : il évite tout conflit sur la machine hôte tout en conservant la communication interne via le réseau Docker.

2.2 Démarrage et diagnostic

Le cluster est démarré par la commande suivante :

```
docker-compose up -d
```

L'état des conteneurs est ensuite contrôlé par :

```
docker-compose ps
```

Le statut `healthy` atteste du succès des healthchecks internes. Pour le NameNode, cela garantit que le service est opérationnel et prêt à gérer les métadonnées HDFS. Les deux DataNodes apparaissent également en état sain, confirmant leur intégration au cluster.

2.3 Validation via l'interface Web

L'interface Web du NameNode (<http://localhost:9870>) permet de visualiser l'état global du cluster. Dans l'onglet `Datanodes`, les deux nœuds `datanode1` et `datanode2` sont listés avec le statut `Live`. Cette observation confirme que le déploiement Docker est fonctionnel et que la communication NameNode–DataNodes est établie.

3. Exercice 2 — Ingestion et consultation des données

3.1 Création de l'espace de stockage HDFS

```
hdfs dfs -mkdir -p /user/hashimi/c21454/datasets
```

La commande `mkdir -p` crée la hiérarchie de répertoires nécessaire. L'option `-p` permet de créer les répertoires parents manquants, conformément au comportement habituel des systèmes de fichiers.

3.2 Chargement du dataset

```
hdfs dfs -put /tp/data/nyctrip.csv /user/hashimi/c21454/datasets/
```

Cette opération transfère le fichier depuis le système de fichiers local (monté dans le conteneur) vers HDFS. Le NameNode enregistre les métadonnées tandis que HDFS découpe automatiquement le fichier en blocs de 128 Mo et les distribue sur les DataNodes disponibles.

3.3 Vérification et facteur de réplication

```
hdfs dfs -ls /user/hashimi/c21454/datasets/  
hdfs dfs -stat "%r" /user/hashimi/c21454/datasets/nyctrip.csv
```

La commande `ls` confirme la présence du fichier `nyctrip.csv` (562 386 202 octets, soit environ 562 Mo). La colonne de réplication affiche la valeur 3, correspondant au facteur par défaut de HDFS. Chaque bloc est ainsi dupliqué sur trois nœuds, garantissant la tolérance aux pannes.

3.4 Modification du facteur de réplication

```
hdfs dfs -setrep -w 2 /user/hashimi/c21454/datasets/nyctrip.csv
```

L'option `-setrep 2` réduit le nombre de copies à deux. L'option `-w` (wait) est essentielle : elle force Hadoop à attendre la fin effective de la réplication avant de retourner le contrôle. Sans cette option, la commande pourrait se terminer prématurément, laissant des blocs partiellement répliqués.

3.5 Synthèse

Cet exercice illustre le cycle complet d'ingestion d'un dataset volumineux dans HDFS et met en évidence le rôle central de la réplication dans la fiabilité du stockage distribué.

4. Exercice 3 — Analyse des blocs

4.1 Diagnostic FSCK

```
hdfs fsck /user/hashimi/c21454/datasets/nyctrip.csv -files -blocks -locations
```

La commande `fsck` (File System Check) analyse l'intégrité du fichier dans HDFS. Le résultat `HEALTHY` confirme que tous les blocs sont présents, correctement répliqués et localisés sur les DataNodes du cluster.

4.2 Découpage en blocs

Avec une taille de bloc par défaut de 128 Mo et un fichier de 562 Mo, le calcul théorique donne : $562 \div 128 \approx 4,4$, soit 5 blocs au total. HDFS ne stocke jamais un fichier volumineux dans un bloc unique : cette fragmentation permet le traitement parallèle, la répartition de charge et la scalabilité horizontale, piliers fondamentaux du paradigme Big Data.

4.3 Localisation et réplication

Chaque bloc est réparti sur plusieurs DataNodes conformément au facteur de réplication configuré. Cette redondance géographique (au sein du cluster) assure la disponibilité des données même en cas de défaillance matérielle ou logicielle d'un nœud.

4.4 Test de tolérance aux pannes

```
docker stop datanode1  
hdfs dfs -cat /user/hashimi/c21454/datasets/nyctrip.csv | head
```

L'arrêt simulé de `datanode1` n'empêche pas la lecture du fichier. Grâce à la réplication, chaque bloc dispose de copies sur d'autres DataNodes. HDFS route automatiquement les requêtes de lecture vers les nœuds disponibles, démontrant la robustesse du système face aux pannes fréquentes en environnement distribué.

5. Exercice 4 — Gestion avancée des flux HDFS

5.1 Fusion de fichiers (getmerge)

Trois fichiers textuels (rapport1.txt, rapport2.txt, rapport3.txt) ont été chargés dans /user/hashimi/c21454/rapports/, puis fusionnés localement via la commande getmerge, qui concatène l'ensemble des fichiers d'un répertoire HDFS en un fichier unique.

```
hdfs dfs -getmerge /user/hashimi/c21454/rapports/ /tmp/fusion_hashimi.txt
```

5.2 Gestion des permissions

Un fichier personnes.txt a été créé sur HDFS, puis ses droits ont été restreints à l'aide de chown et chmod 600. Cette manipulation rappelle que HDFS implémente un modèle de permissions comparable à POSIX (propriétaire, groupe, autres), indispensable en contexte multi-utilisateurs.

```
hdfs dfs -chown root:supergroup /user/hashimi/c21454/datasets/personnes.txt
hdfs dfs -chmod 600 /user/hashimi/c21454/datasets/personnes.txt
```

5.3 Ajout incrémental via appendToFile

```
echo "[2026-06-14] INFO: Log Hashimi - taxi enregistre" > log_hashimi.txt
hdfs dfs -appendToFile log_hashimi.txt /user/hashimi/c21454/datasets/personnes.txt
```

La commande appendToFile permet d'ajouter du contenu à un fichier HDFS existant sans le réimporter intégralement. Cette fonctionnalité est particulièrement utile pour les flux de logs et les données séquentielles. L'expérimentation a également mis en lumière l'importance de la précision des chemins : une erreur de nommage peut créer un fichier distinct plutôt que d'enrichir le fichier cible. Par ailleurs, les opérations d'écriture nécessitent des DataNodes opérationnels ; une indisponibilité temporaire peut provoquer des erreurs de pipeline (lease recovery).

6. Exercice 5 — Clôture et maintenance du cluster

6.1 Arrêt contrôlé

```
docker-compose stop  
# ou  
docker-compose down
```

L'arrêt via `docker-compose stop` interrompt les conteneurs tout en préservant les volumes Docker. Les données HDFS demeurent intactes et seront disponibles au prochain redémarrage.

6.2 Persistance des données

Les blocs HDFS sont stockés dans des volumes Docker persistants. Tant que l'option `-v` n'est pas utilisée, un redémarrage du cluster ne provoque aucune perte de données. En revanche, la commande `docker-compose down -v` supprime les volumes associés, entraînant un effacement irréversible du contenu HDFS.

6.3 Commandes à risque en production

```
docker-compose down -v  
docker system prune -a --volumes
```

Ces commandes sont particulièrement dangereuses en environnement de production Big Data. Elles suppriment conteneurs, images et volumes, détruisant ainsi l'ensemble des données distribuées. Toute opération de maintenance doit être précédée d'une sauvegarde et d'une validation explicite des volumes concernés.

7. Tableau récapitulatif des concepts Hadoop

Concept	Définition et observation durant le TP
NameNode	Nœud maître gérant les métadonnées HDFS (arborescence, blocs, permissions). Interface Web sur le p
DataNode	Nœuds esclaves stockant physiquement les blocs. Assurent lecture, écriture et réplication.
Block Size	Taille maximale d'un bloc (128 Mo par défaut). Le fichier nyctrip.csv (562 Mo) est découpé en 5 bloc
Replication Factor	Nombre de copies par bloc (3 par défaut, modifié à 2). Garantit la tolérance aux pannes.
FSCCK	Outil de diagnostic vérifiant l'intégrité, la localisation et l'état de santé des fichiers HDFS.
appendToFile	Commande d'ajout incrémental au contenu d'un fichier HDFS sans réimport complet.
getmerge	Fusion de fichiers d'un répertoire HDFS en un fichier local unique.

8. Conclusion générale

Ce travail pratique a permis de mettre en œuvre et d'analyser les composants essentiels de l'écosystème Hadoop. Le déploiement containerisé via Docker Compose facilite la reproductibilité de l'environnement expérimental, tandis que l'ingestion du dataset NYC Taxi Trip offre un cas d'usage réaliste de traitement de données massives.

Les exercices réalisés démontrent que HDFS repose sur trois principes architecturaux fondamentaux : la fragmentation des fichiers en blocs, la réplication pour la fiabilité, et la distribution sur plusieurs nœuds pour la scalabilité. La simulation de panne et l'analyse FSCCK confirment la robustesse du système. Enfin, les opérations avancées (fusion, permissions, append) et la gestion du cycle de vie du cluster soulignent l'importance d'une administration rigoureuse en contexte Big Data.

Références bibliographiques

- Apache Hadoop Documentation — HDFS Architecture. <https://hadoop.apache.org/docs/stable/>
- White, T. (2015). Hadoop: The Definitive Guide (4th ed.). O'Reilly Media.
- Docker Inc. — Docker Compose Documentation. <https://docs.docker.com/compose/>
- Big Data Europe — bde2020/hadoop-docker images. <https://github.com/big-data-europe/docker-hadoop>